

WorkFlow — Project Context

Checkpoint — Final da Aula 2

1. Objetivo do projeto

O **WorkFlow** será uma plataforma genérica e multiempresa para gerenciamento de projetos, tarefas, equipes, comunicação interna e acompanhamento de atividades.

O projeto possui dois objetivos simultâneos:

1. Ser utilizado de forma real por empresas para organização de seus projetos.
2. Servir como projeto profissional de portfólio demonstrando conhecimentos exigidos em vagas .NET/React.

O sistema não possui relação com Horus-W ou outros sistemas existentes.

2. Metodologia de desenvolvimento

O desenvolvimento está sendo conduzido como uma aula.

Fluxo:

1. Explicação do conceito.
2. Explicação do motivo da decisão.
3. Definição da arquitetura/regra.
4. Implementação pelo aluno.
5. Revisão.
6. Testes.
7. Git.
8. Documentação.

O objetivo não é simplesmente fornecer código pronto, mas ensinar o motivo das decisões.

3. Stack definida

Backend

- C#
- .NET 10
- ASP.NET Core Web API
- Entity Framework Core
- Npgsql
- PostgreSQL

Dapper não será utilizado.

O objetivo é aprender EF Core e o acesso a banco de dados dentro do ecossistema .NET.

Posteriormente também será estudado ADO.NET para entender as abstrações utilizadas pelo EF Core.

Frontend

- React
- TypeScript
- Vite

Posteriormente serão utilizadas bibliotecas apropriadas conforme necessidade.

Tempo real

- SignalR

Testes

- xUnit
- testes unitários
- testes de integração

Infraestrutura futura

- Docker
- Docker Compose
- GitHub Actions
- CI/CD
- deploy em cloud

Tecnologias futuras, somente se justificadas

- Redis
- MongoDB

Não adicionar tecnologias somente para enriquecer currículo.

4. Arquitetura planejada

```
WorkFlow/  
|  
├─ src/  
|   ├─ WorkFlow.API/  
|   ├─ WorkFlow.Application/  
|   └─ WorkFlow.Domain/
```

```

|   └─ Workflow.Infrastructure/
|
|   └─ tests/
|       ├── Workflow.UnitTests/
|       └─ Workflow.IntegrationTests/
|
|   └─ frontend/
|   └─ docs/
|   └─ Workflow.sln

```

Responsabilidades:

Domain

→ entidades e regras centrais do negócio

Application

→ casos de uso e coordenação da aplicação

Infrastructure

→ EF Core, PostgreSQL e serviços externos

API

→ HTTP, autenticação, autorização e entrada da aplicação

A camada Domain não deve conhecer EF Core, PostgreSQL, HTTP ou React.

5. Multi-tenancy

O Workflow será **multiempresa**.

Cada empresa será um **Tenant**.

```

Workflow
├── Tenant A
│   ├── Users
│   ├── Projects
│   └─ Chat
└── Tenant B
    ├── Users
    ├── Projects
    └─ Chat

```

Dados de um Tenant jamais poderão ser acessados por usuários de outro Tenant.

O isolamento deverá obrigatoriamente ser validado no backend.

Esconder informações no React não é considerado mecanismo de segurança.

6. Identificadores

Foi discutida a utilização de tokens aleatórios em vez de IDs para dificultar enumeração.

Decisão:

- relacionamentos internos continuarão utilizando Foreign Keys;
- identificadores não devem ser utilizados como mecanismo de autorização;
- poderá existir um `PublicId` não sequencial para exposição externa;
- descobrir um `PublicId` nunca concede acesso ao recurso.

Conceitualmente:

```
Id
→ identificação/relacionamento interno

PublicId
→ identificação externa
```

A escolha entre `int`, `long`, `Guid`, UUID ou ULID ainda não foi realizada.

7. Segurança

Senhas de usuários

Nunca armazenar senha original ou senha criptografada reversível.

Persistir somente:

```
PasswordHash
```

utilizando mecanismo seguro de hashing de senha.

Credenciais de infraestrutura

Não poderão ficar:

- hardcoded;
- versionadas no Git;

- dentro do código-fonte com valores reais.

Serão utilizados posteriormente:

- User Secrets;
- environment variables;
- mecanismos de secrets em produção.

Autorização

Deverá considerar, conforme a operação:

```
Usuário autenticado
+
Tenant
+
Role global
+
Project Membership
+
Project Permissions
+
Responsabilidade pelo recurso
+
Estado do domínio
```

8. Roles globais

Foram definidos quatro papéis:

```
SYSTEM_ADMIN
TENANT_ADMIN
PROJECT_MANAGER
MEMBER
```

SYSTEM_ADMIN

Administrador da plataforma WorkFlow.

Não representa normalmente um funcionário de um Tenant.

TENANT_ADMIN

Administrador de uma empresa específica.

Pode administrar usuários e projetos de seu próprio Tenant.

PROJECT_MANAGER

Gerencia projetos nos quais possui atribuição administrativa.

Ser PROJECT_MANAGER não concede administração automática de todos os projetos do Tenant.

MEMBER

Usuário comum.

Executa tarefas, comenta e participa dos projetos aos quais foi adicionado.

Pode receber permissões adicionais por projeto.

9. Autorização por projeto

Além do Role global existirão permissões ligadas ao `ProjectMember`.

Permissões iniciais candidatas:

```
EDIT_PROJECT
MANAGE_PROJECT_MEMBERS
MANAGE_PROJECT_PERMISSIONS
COMPLETE_PROJECT
REOPEN_PROJECT
ARCHIVE_PROJECT

CREATE_TASK
EDIT_TASK
CLAIM_TASK
ASSIGN_TASK
MANAGE_TASK_COLLABORATORS
CANCEL_TASK
REOPEN_TASK

VALIDATE_TASK
SELF_VALIDATE_TASK
```

Não utilizar dezenas de propriedades booleanas diretamente dentro de `User`.

Entidade conceitual:

ProjectMemberPermission

- Id
- ProjectMemberId
- Permission
- GrantedAt
- GrantedByUserId
- RevokedAt
- RevokedByUserId

Permissão revogada pode permanecer armazenada para histórico.

10. Auto-validação

Foi descartada a regra de que o responsável nunca pode validar sua própria tarefa.

O sistema precisa funcionar também em empresas pequenas.

Foram definidos dois conceitos:

VALIDATE_TASK

→ pode validar tarefas

SELF_VALIDATE_TASK

→ pode validar uma tarefa da qual é o próprio responsável

No MVP:

- TENANT_ADMIN poderá auto-validar;
- PROJECT_MANAGER responsável pela gestão poderá auto-validar;
- MEMBER poderá receber essa permissão explicitamente.

Mesmo em auto-validação, o fluxo deverá continuar:

IN_PROGRESS

→ VALIDATION

→ DONE

A pessoa não pula diretamente de `IN_PROGRESS` para `DONE`.

11. Entidades principais identificadas

```
Tenant
User
Project
ProjectMember
ProjectMemberPermission
Task
TaskCollaborator
TaskComment
ChatMessage
Notification
TaskHistory
ProjectHistory
```

Ainda poderão surgir outras entidades durante o desenvolvimento.

12. Tenant

Estrutura conceitual atual:

```
Tenant
- Id
- PublicId
- Name
- RegistrationNumber
- Email
- Phone
- IsActive
- CreatedAt
- UpdatedAt
```

`RegistrationNumber` foi preferido conceitualmente em vez de amarrar o domínio exclusivamente a CNPJ/CPF.

13. User

Estrutura conceitual:

```
User
- Id
- PublicId
- TenantId
```


- Name
- Email
- PasswordHash
- Role
- IsActive
- CreatedAt
- UpdatedAt

SYSTEM_ADMIN poderá ser uma exceção sem Tenant.

Usuários comuns deverão pertencer a um Tenant.

Usuários desativados não serão fisicamente removidos, preservando histórico e autoria.

14. Project

Estrutura conceitual atual:

```
Project
- Id
- PublicId
- TenantId
- Name
- Description
- ResponsibleUserId
- CreatedByUserId
- Status
- DueDate
- CreatedAt
- UpdatedAt
- ArchivedAt
```

`ResponsibleUserId` é opcional.

O criador do projeto não é necessariamente seu responsável nem proprietário permanente.

15. ProjectMember

Representa participação em um projeto.

```
ProjectMember
- Id
- ProjectId
- UserId
```

- AddedAt
- AddedByUserId
- RemovedAt

Um mesmo usuário não poderá possuir duas participações ativas no mesmo projeto.

Remoção não precisa apagar fisicamente o histórico da participação.

16. Status de Project

PLANNING
IN_PROGRESS
PAUSED
COMPLETED
ARCHIVED

Fluxo principal:

PLANNING → IN_PROGRESS
IN_PROGRESS → PAUSED
PAUSED → IN_PROGRESS
IN_PROGRESS → COMPLETED
COMPLETED → IN_PROGRESS
COMPLETED → ARCHIVED
PLANNING → ARCHIVED

`COMPLETED → IN_PROGRESS` representa reabertura.

Reabertura exige justificativa.

Projeto arquivado deverá ser restaurado antes de voltar à operação.

`RestoreProject()` e `ReopenProject()` são conceitos diferentes.

17. Projeto PAUSED

Pausar projeto:

- exige motivo;
- não altera automaticamente os estados das Tasks;
- bloqueia operações de execução enquanto o projeto permanecer pausado;
- não altera automaticamente o prazo.

Ao retomar:

- manter prazo atual
- alterar prazo

Se alterar, o usuário deverá selecionar explicitamente a nova data.

18. Conclusão de Project

Um projeto só poderá ser **COMPLETED** se todas as suas Tasks estiverem:

DONE
ou
CANCELLED

Não permitir projeto **COMPLETED** com tarefas abertas.

Projeto concluído poderá ser reaberto mediante justificativa.

19. Arquivamento de Project

Projeto não será normalmente excluído.

Arquivamento mantém:

- dados;
- histórico;
- tarefas;
- informações para relatórios.

Projeto **ARCHIVED** deverá ficar essencialmente read-only até ser restaurado.

20. Task

Estrutura conceitual atual:

Task

- Id
- PublicId
- ProjectId
- Title
- Description
- Status

- Priority
- ResponsibleUserId
- ValidatorUserId
- DueDate
- CreatedByUserId
- CreatedAt
- UpdatedAt
- ArchivedAt

Uma Task poderá existir sem responsável.

Uma Task pode ter somente um responsável principal.

Pode possuir vários colaboradores.

21. TaskPriority

LOW
MEDIUM
HIGH
CRITICAL

22. TaskStatus

BACKLOG
TODO
IN_PROGRESS
PAUSED
VALIDATION
DONE
CANCELLED

23. Máquina de estados da Task

Fluxo principal:

BACKLOG → TODO
TODO → IN_PROGRESS

```
IN_PROGRESS → VALIDATION
VALIDATION → DONE
```

Outras transições:

```
BACKLOG → CANCELLED

TODO → PAUSED
TODO → CANCELLED

IN_PROGRESS → PAUSED
IN_PROGRESS → CANCELLED

VALIDATION → IN_PROGRESS
(rejeição ou retirada da validação)

PAUSED → estado operacional anterior

DONE → IN_PROGRESS
(reabertura)

CANCELLED → TODO
(reabertura)
```

Não permitir alterações arbitrárias de `Status`.

O objetivo futuro é utilizar comportamentos como:

```
Start()
Pause()
Resume()
SendToValidation()
Approve()
Reject()
Cancel()
Reopen()
```

em vez de simplesmente:

```
task.Status = ...
```

24. Responsável da Task

`ResponsibleUserId` poderá ser `null`.

Uma tarefa sem responsável poderá ser assumida por membros autorizados com `CLAIM_TASK`.

Uma tarefa `IN_PROGRESS` ou `VALIDATION` deverá possuir responsável.

Se o responsável for removido/desativado enquanto a tarefa estiver `IN_PROGRESS`:

```
ResponsibleUserId = null  
IN_PROGRESS → TODO
```

A alteração será registrada no histórico e pessoas responsáveis pela gestão serão notificadas.

25. TaskCollaborator

```
TaskCollaborator  
- Id  
- TaskId  
- UserId  
- AddedAt  
- AddedByUserId  
- RemovedAt
```

Não armazenar `ProjectId`, pois o projeto pode ser determinado através de:

```
Task → Project
```

Colaborador deve ser membro do projeto.

Responsável principal não deverá simultaneamente aparecer como colaborador.

26. Validação de Task

Ao terminar o trabalho, o responsável envia:

```
IN_PROGRESS → VALIDATION
```

Membro comum não conclui diretamente como `DONE`.

Tarefa em `VALIDATION` poderá possuir:

```
ValidatorUserId
```

Se:

```
ValidatorUserId = null
```

a validação está aguardando alguém assumir.

Usuários autorizados poderão realizar:

```
ClaimValidation()
```

Quando alguém assume:

```
ValidatorUserId = usuário
```

Apenas um validador poderá existir por vez.

27. Operações de validação

Foram definidos conceitualmente:

```
ClaimValidation  
ReleaseValidation  
ReassignValidation  
WithdrawFromValidation  
ApproveValidation  
RejectValidation
```

O validador poderá liberar a validação.

Usuário administrativo autorizado poderá transferi-la.

Se a Task deixar `VALIDATION`, `ValidatorUserId` deverá ser limpo.

Todas essas operações deverão aparecer no histórico.

28. Retirada pelo próprio responsável

O responsável que enviou a Task para `VALIDATION` poderá perceber um erro e retirá-la antes da aprovação.

Fluxo:

`VALIDATION` → `IN_PROGRESS`

Exige motivo.

Isso é diferente de rejeição pelo validador.

Ações distintas:

`VALIDATION_WITHDRAWN`
`VALIDATION_REJECTED`

29. Prazo da Task

Pausar tarefa não altera automaticamente `DueDate`.

Ao retomar:

- manter prazo atual
- alterar prazo

Se selecionar alteração, o usuário escolhe a nova data explicitamente.

A mudança de prazo deverá ser registrada no histórico.

30. PAUSED da Task

Pausa exige motivo.

Ao retomar, a tarefa deve retornar ao estado operacional anterior válido.

A forma exata de persistir o estado anterior ainda será definida durante implementação.

31. DONE e CANCELLED

Tasks **DONE** e **CANCELLED** poderão continuar recebendo comentários.

Uma Task **DONE** poderá ser reaberta por usuário autorizado:

DONE → IN_PROGRESS

com justificativa.

Task **CANCELLED** poderá ser reaberta:

CANCELLED → TODO

com justificativa.

Uma Task não poderá ser reaberta se seu projeto estiver **COMPLETED** ou **ARCHIVED**.

Primeiro deverá ser reaberto/restaurado o Project.

32. TaskComment

Estrutura conceitual:

```
TaskComment
- Id
- TaskId
- AuthorUserId
- Content
- CreatedAt
- UpdatedAt
- DeletedAt
- DeletedByUserId
```

Comentários e histórico são conceitos diferentes.

Comentários representam comunicação humana.

Histórico representa acontecimentos do sistema.

Ainda não foi definido o tempo exato para edição/exclusão de comentários de Task.

33. ChatMessage

Cada Tenant terá inicialmente um chat geral.

```
ChatMessage
- Id
- TenantId
- AuthorUserId
- Content
- CreatedAt
- UpdatedAt
- ExpiresAt
- DeletedAt
- DeletedByUserId
```

Mensagens permanecem disponíveis por 7 dias.

Autor poderá editar/excluir sua própria mensagem durante **5 minutos após o envio**.

Depois disso fica bloqueada para alteração normal.

Mensagens antigas poderão ser removidas posteriormente por processamento em background.

34. Chat futuro

Fora do MVP inicial:

- chat privado;
 - chat por projeto;
 - grupos privados;
 - anexos.
-

35. Notification

Estrutura conceitual:

```
Notification
- Id
- RecipientUserId
- ActorUserId
- Type
- Title
- Message
```

- ResourceType
- ResourceId
- CreatedAt
- ReadAt
- DismissedAt

`ActorUserId` poderá ser nulo em notificações automáticas do sistema.

Preferir `ReadAt` a `IsRead`.

Preferir `DismissedAt` a `IsDismissed`.

36. Persistência das notificações

SignalR não é a notificação.

Fluxo obrigatório:

```
Evento
↓
Notification persistida no banco
↓
tentativa de envio em tempo real
```

Usuário offline deve receber a notificação quando entrar posteriormente.

SignalR é somente mecanismo de entrega em tempo real.

37. Retenção de Notification

Notificações poderão permanecer armazenadas por até 90 dias.

Usuário poderá:

- remover notificações individualmente;
- limpar sua lista quando quiser.

Isso afeta somente o histórico daquele destinatário.

Remoção visual poderá utilizar `DismissedAt`.

38. Política de notificações

Usuário que executou a própria ação normalmente não recebe notificação sobre ela.

Eventos relacionados ao **Project inteiro**:

notificam todos os demais membros ativos do Project.

Exemplos:

- projeto pausado;
- projeto retomado;
- projeto concluído;
- projeto reaberto;
- prazo do projeto alterado;
- responsável do projeto alterado;
- arquivamento.

Eventos relacionados a uma **Task**:

normalmente notificam somente os participantes da Task.

Participantes:

```
ResponsibleUser  
+  
TaskCollaborators
```

Exemplo:

Prazo de Task alterado:

```
responsável + colaboradores
```

Prazo de Project alterado:

```
todos os membros do Project
```

39. Exceções de notificação da Task

Eventos que precisam de ação de outros usuários podem incluir terceiros relevantes.

Exemplo:

Task enviada para **VALIDATION**:

```
participantes
+
usuários autorizados a validar
```

Quando alguém assume uma validação, participantes poderão ser avisados e as telas dos demais validadores serão atualizadas em tempo real.

40. Activity Feed x Notification x History

Três conceitos diferentes:

History

Registro auditável do que aconteceu com determinado recurso.

Activity Feed

Visão geral de acontecimentos recentes do Project.

Pode mostrar atividades mesmo para membros que não receberam notificação.

Notification

Aviso dirigido a alguém que precisa saber daquele evento.

41. TaskHistory

Estrutura conceitual inicial:

```
TaskHistory
- Id
- TaskId
- ActorUserId
- Action
- Reason
- OldValue
- NewValue
- CreatedAt
```

Exemplos de ações:

```
CREATED
ASSIGNED
STARTED
PAUSED
RESUMED
SENT_TO_VALIDATION
VALIDATION_CLAIMED
VALIDATION_RELEASED
VALIDATION_REASSIGNED
VALIDATION_WITHDRAWN
VALIDATION_APPROVED
VALIDATION_REJECTED
DUE_DATE_CHANGED
PRIORITY_CHANGED
CANCELLED
REOPENED
ARCHIVED
```

Histórico deve ser tratado conceitualmente como **append-only**.

Usuários comuns não podem editar histórico.

Administradores também não devem reescrever silenciosamente o passado.

42. ProjectHistory

Conceitualmente:

```
ProjectHistory
- Id
- ProjectId
- ActorUserId
- Action
- Reason
- OldValue
- NewValue
- CreatedAt
```

Ações possíveis:

```
CREATED
STARTED
PAUSED
RESUMED
COMPLETED
REOPENED
```

```
ARCHIVED
RESTORED
DUE_DATE_CHANGED
RESPONSIBLE_CHANGED
MEMBER_ADDED
MEMBER_REMOVED
PERMISSION_GRANTED
PERMISSION_REVOKED
```

43. Eventos

Eventos importantes do domínio/aplicação poderão produzir consequências.

Exemplo:

```
TaskDueDateChanged
```

poderá gerar:

```
TaskHistory
Notification
SignalR
```

Uso de eventos não significa microservices.

O projeto começará como **monólito modular**.

Não utilizar RabbitMQ/Kafka/microservices sem necessidade concreta.

44. Persistência e transações

Operações críticas deverão preservar consistência.

Exemplo:

```
alterar Task
+
registrar History
+
criar Notification
```

devem ser persistidos de forma consistente.

Se algo crítico falhar:

ROLLBACK

SignalR deverá ocorrer **após a persistência bem-sucedida**.

Princípio:

Primeiro persistimos a verdade; depois avisamos o mundo externo.

45. Relacionamentos principais

Tenant 1:N User

Tenant 1:N Project

Tenant 1:N ChatMessage

Project 1:N Task

Project N:N User
através de ProjectMember

Task N:N User
através de TaskCollaborator

Task 1:N TaskComment

Task 1:N TaskHistory

Project 1:N ProjectHistory

User 1:N Notification
como destinatário

Vários relacionamentos também apontam para `User` com significados diferentes:

CreatedByUser
ResponsibleUser
ValidatorUser
ActorUser
AuthorUser
DeletedByUser
AddedByUser

Esses relacionamentos deverão ser explicitamente configurados no EF Core posteriormente.

46. Exclusão de dados

Política conservadora.

Dados importantes não devem depender de cascade delete destrutivo.

Preferência:

```
Tenant → desativar
User → desativar
Project → arquivar
Task → arquivar
```

Dados temporários:

```
ChatMessage → retenção 7 dias
Notification → retenção 90 dias
```

podem futuramente ser removidos fisicamente.

47. Dashboard

O dashboard deverá ser a principal visão diária do usuário.

Deverá responder rapidamente:

O que preciso fazer hoje?

O que aconteceu?

O que está atrasado?

Quais projetos precisam da minha atenção?

Exibir futuramente:

- minhas tarefas;
- tarefas atrasadas;
- tarefas próximas do prazo;
- projetos;
- atividades recentes;

- notificações.
-

48. Uso contínuo

O WorkFlow foi pensado para permanecer aberto durante o expediente.

No futuro poderá iniciar automaticamente com o computador.

Primeira versão:

Web + React + SignalR

Notificações nativas do Windows e comportamento desktop ficam para etapa final do projeto.

49. Funcionalidades futuras já identificadas

Não implementar inicialmente:

- subtarefas;
 - dependências estruturadas entre tarefas;
 - chat por projeto;
 - chat privado;
 - anexos;
 - notificações nativas do Windows;
 - cliente desktop;
 - aplicativo mobile;
 - Redis;
 - MongoDB;
 - integrações externas.
-

50. Próxima etapa

As Aulas 1 e 2 estão concluídas.

A próxima aula será:

Aula 3 — Criação da Solution e arquitetura .NET

Começar do zero no Visual Studio.

Criar:

```
Workflow
|
|— src/
|   |— Workflow.Domain
|   |— Workflow.Application
|   |— Workflow.Infrastructure
|   |— Workflow.API
|
|— tests/
|   |— Workflow.UnitTests
|   |— Workflow.IntegrationTests
|
|— docs/
```

Explicar antes de implementar:

- tipos de projeto .NET;
- Class Library;
- Web API;
- referências entre projetos;
- direção das dependências;
- por que Domain não depende de Infrastructure;
- Dependency Injection;
- onde EF Core entra;
- onde PostgreSQL entra;
- onde React ficará.

Depois da estrutura, criar a primeira entidade real em C#, começando pelo núcleo do domínio antes de configurar EF Core.